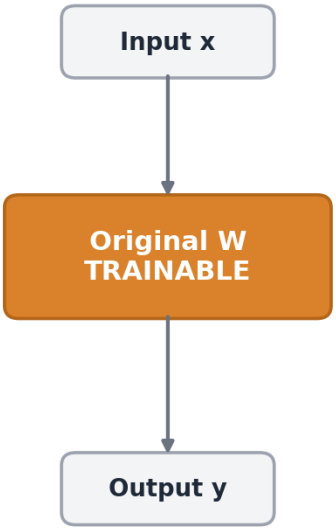
How LoRA changes training

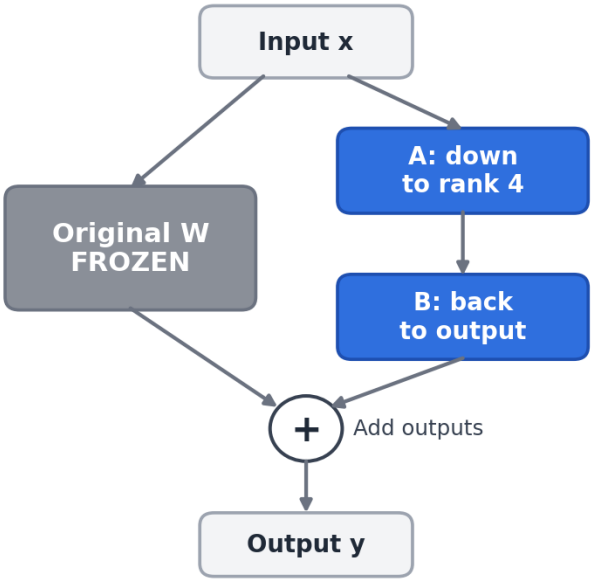
Same pretrained starting weights and the same new training data

FULL FINE-TUNING



`y = Wx`

LoRA FINE-TUNING



`y = Wx + (alpha / rank) BAx`

Both approaches learn from the same sports-team question-and-answer examples. Full fine-tuning lets every original model parameter change. LoRA freezes the original parameters and learns small added matrices. The diagram shows one adapted linear projection; the experiment applies adapters throughout the transformer.

Only the blue A and B matrices train in the LoRA model. Rank = 4; alpha = 8 (scaling alpha / rank = 2). Biases are omitted from this conceptual diagram. The trainable count of 589,824 equals the arithmetic for rank-4 adapters on the QKV, attention-output, and both MLP projections in all 12 GPT-2 blocks (49,152 values per block x 12). The report itself records only the rank and alpha, not the module list.

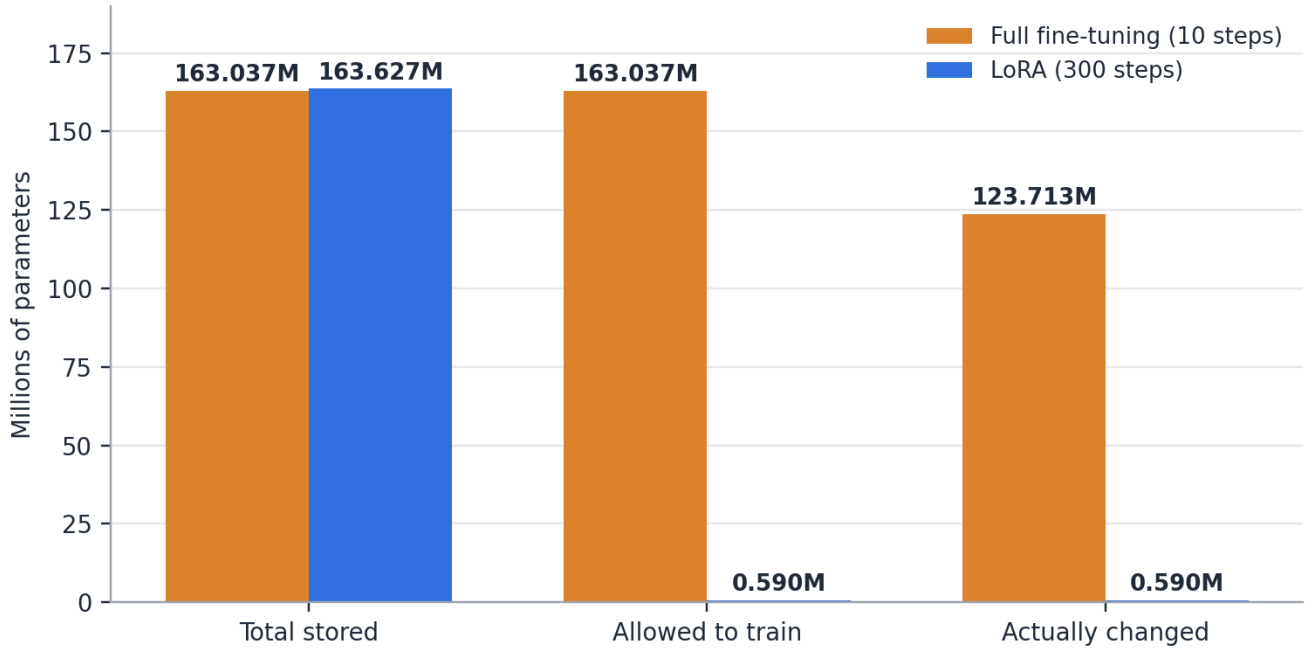
163,037,184	589,824
trainable in full fine-tuning	trainable with LoRA

The untouched pretrained model is a third reference: it receives no task training. Both trained models use the same examples, sample order, batch size, learning rate and seed, but **not the same number of steps**: LoRA ran 300 steps and full fine-tuning ran 10 (see the limits on page 7).

Source: saved experiment reports (report.json, report_1.json) | V1

Total, trainable, and actually changed

These are different quantities; all counts refer to individual scalar values



Total: every model parameter, including frozen weights. **Trainable:** parameters allowed to receive optimizer updates. **Actually changed:** individual values differing from their pre-training snapshots at the final step. Being trainable does not guarantee a value changed.

276.4x	0	75.9%
fewer trainable parameters with LoRA	frozen base values changed by LoRA	of full fine-tuning values changed after only 10 steps

LoRA makes 0.3605% of its total parameters trainable. Full fine-tuning changed 123,712,795 of 163,037,184 values; LoRA changed all 589,824 adapter values. The chart uses a zero baseline; the small adapter bars are labeled explicitly.

LoRA audit log (seed 42)

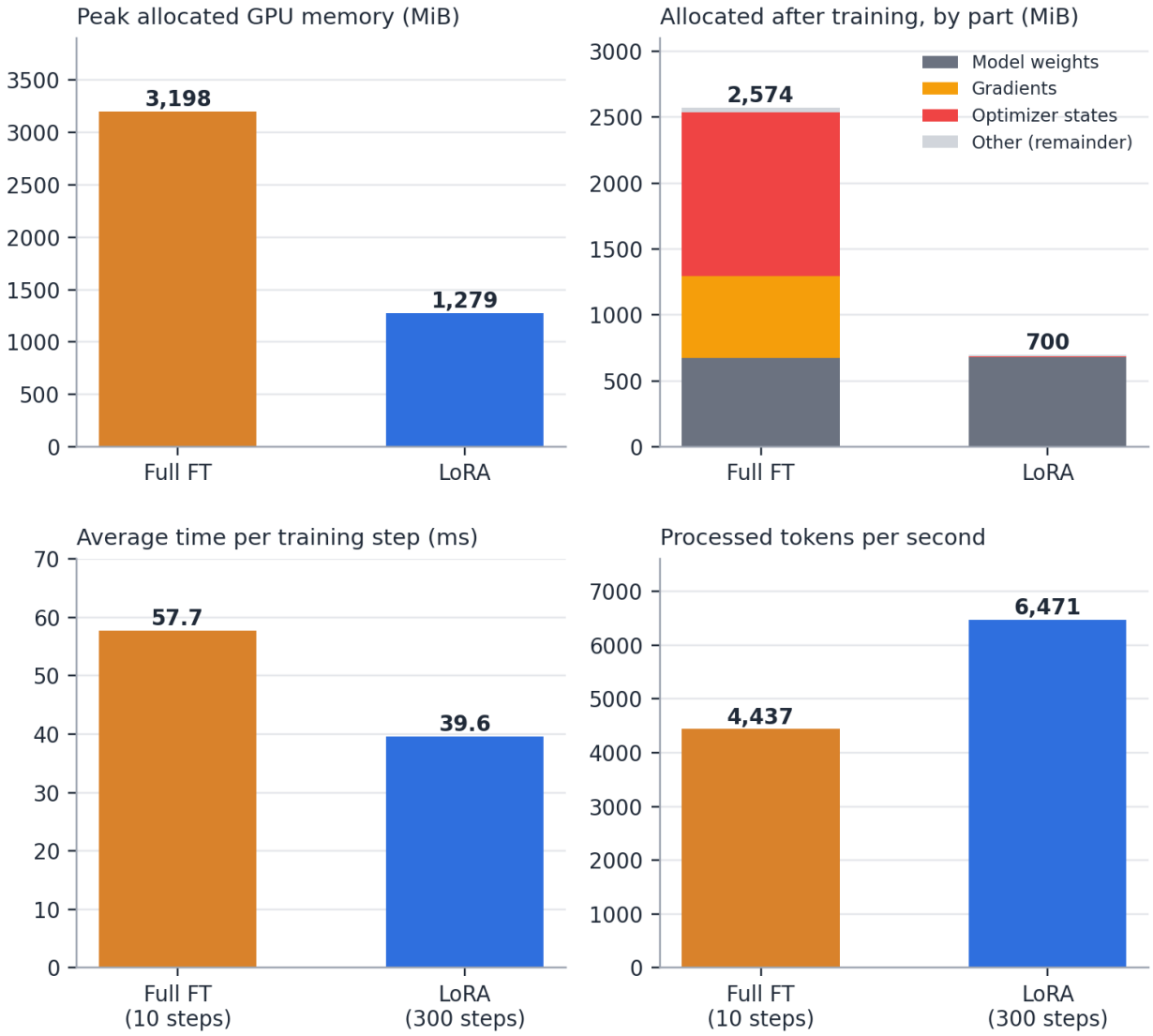
Step	Training loss	Adapter values changed	Frozen base values changed
1	3.854302	0	0
50	0.264965	589,824	0
100	0.025177	589,824	0
150	0.003047	589,824	0
200	0.000560	589,824	0
250	0.000744	589,824	0
300	0.000488	589,824	0

Every logged audit, from step 1 to step 300, found zero changed values among the 163,037,184 frozen base parameters. Step 1 is logged before the first update, so no values differ yet.

Source: saved experiment reports (report.json, report_1.json) | V2

Training resources: what is saved?

NVIDIA A100-SXM4-80GB | FP16 autocast | batch 4 | sequence length 64 | seed 42



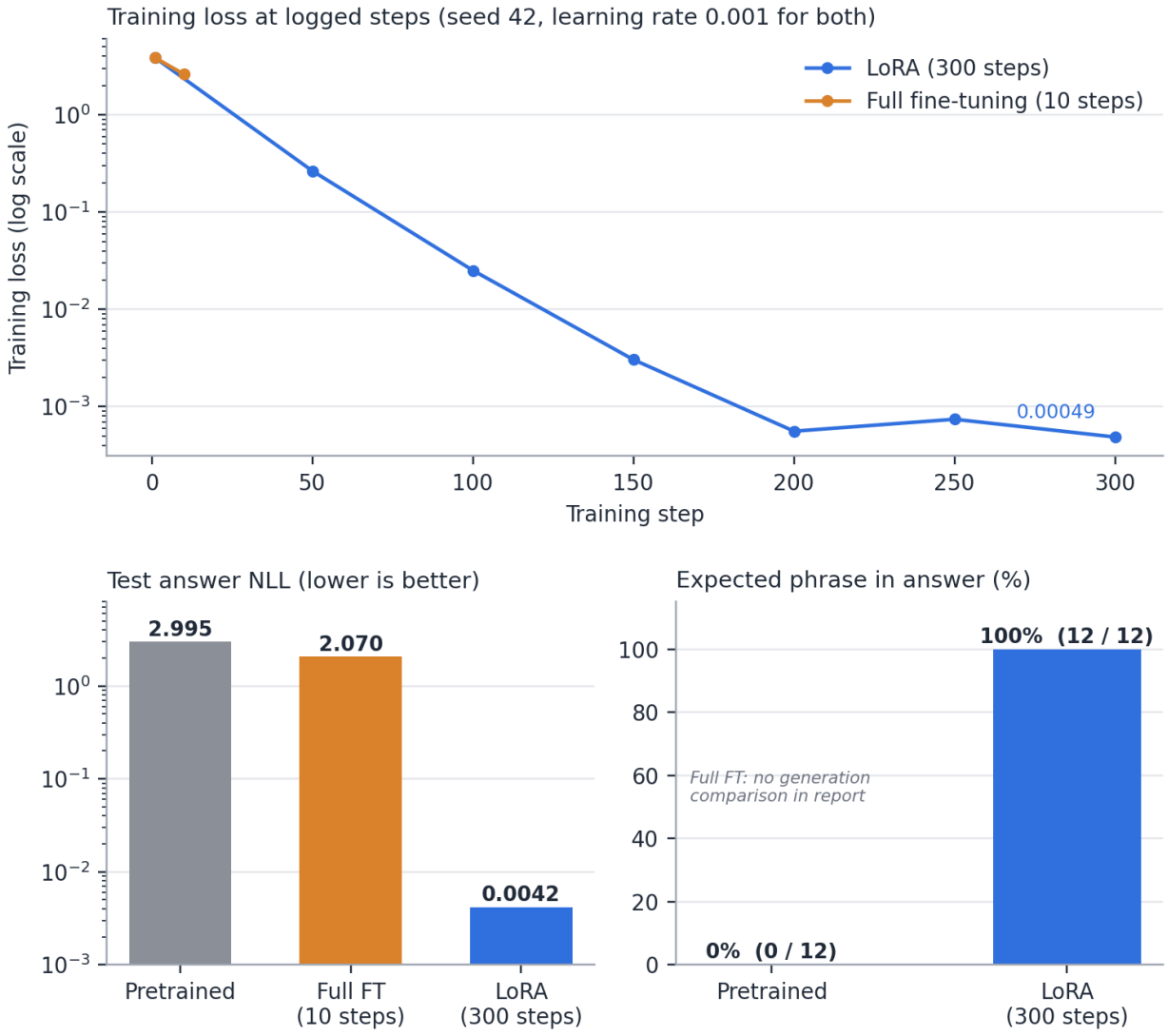
In this run LoRA reduced peak allocated training memory by **60.0%** (3,197.8 MiB to 1,278.8 MiB). The saving comes from gradients and optimizer states: full fine-tuning held about 621.9 MiB of gradients and 1,243.9 MiB of optimizer tensors, versus 2.2 MiB and 4.5 MiB for LoRA. The right-hand upper panel is derived by subtraction; "other" is whatever allocated memory remains after model, gradient and optimizer tensors are removed.

Timing caution: the two runs have different lengths (10 vs 300 steps), and each timing includes its cold first step, which weighs far more in the 10-step run. The per-step and throughput panels are therefore indicative, not a controlled speed comparison.

Source: saved experiment reports (report.json, report_1.json) | V3

Quality alongside parameter efficiency

Loss, test-answer NLL and expected-phrase accuracy | seed 42



After 300 steps LoRA reached a test answer NLL of 0.0042 (pretrained: 2.995) and produced the expected answer phrase for all 12 test questions; the pretrained model produced it for none. Full fine-tuning ran only 10 steps at the same learning rate and reached 2.070; its loss was still falling, so this is not a converged result.

These are synthetic, format-sensitive question-answering scores on 12 questions, not broad conversational quality. Full fine-tuning has no generation comparison in its report, so no accuracy is shown for it. Single seed; no error bars.

Source: saved experiment reports (report.json, report_1.json) | V4

LoRA versus full fine-tuning

Preliminary FP16 results from a single-seed run

Purpose and comparison

Measure whether LoRA can learn a new task while training fewer parameters and using less GPU memory. We compare an unchanged pretrained reference, a short full fine-tuning run (all original weights trainable), and a LoRA run (only added adapters trainable), all on the same new data.

Experiment configuration

Pretrained GPT-2-sized model with 163,037,184 base parameters (separate input and output embeddings). Synthetic sports-team data: question-and-answer pairs about practice location, practice schedule, coach and jersey color for fictional teams. Training file SHA-256 d8cfd1c261e3..., test file 5e71ce22c644... (identical in both runs). The test set has 12 questions covering three teams (Harbor Comets, Pine Falcons, Summit Owls).

Training protocol

NVIDIA A100-SXM4-80GB; torch 2.14.0+cu130, CUDA 13.0; FP16 autocast with TF32 disabled; parameters stored at 4 bytes per value. Batch size 4, padded sequence length 64, learning rate 0.001, seed 42, gradient clipping, and an Adam-style optimizer (two state tensors per trainable parameter, per the recorded optimizer bytes). LoRA rank 4, alpha 8. Generation was capped at 40 new tokens. **Step counts differ:** LoRA 300 steps, full fine-tuning 10 steps. No learning-rate search or validation split is recorded for these runs.

How this differs from the earlier matched FP32 experiment

Setting	Earlier FP32 report	This FP16 run
GPU / precision	NVIDIA A30, FP32	NVIDIA A100-SXM4-80GB, FP16 autocast
Task	Synthetic basketball matches (winners, halftime, comebacks)	Synthetic sports-team facts (location, schedule, coach, jersey)
Batch / length	8 / 128	4 / 64
Steps	1,200 for both methods	300 (LoRA) and 10 (full)
Seeds	42, 43, 44	42 only
Learning rate	Selected per method on validation loss	0.001 for both, no selection

Parameter counts

Measurement	Full fine-tuning	LoRA
Total parameters	163,037,184	163,627,008
Trainable parameters	163,037,184	589,824
Trainable fraction	100%	0.3605%
Changed scalar values (final)	123,712,795 (75.88%)	589,824
Frozen base values changed	Not applicable	0

How the frozen-weight claim was verified

Parameters were compared with snapshots of their initial values at every audit (every 50 steps). Exact scalar comparisons distinguish values that actually changed from those merely allowed to change. All LoRA audits found zero original weight values changed, and the report's base-response check (base_responses_match_initial) is true. LoRA has 276.4 times fewer trainable parameters.

Source: saved run reports report.json (LoRA) and report_1.json (full fine-tuning). 1

Memory and training time

Measured resource results

Measurement	Full fine-tuning (10 steps)	LoRA (300 steps)
FP32-width parameter storage	621.94 MiB	624.19 MiB
Peak allocated GPU memory	3,197.8 MiB	1,278.8 MiB
Peak reserved GPU memory	3,604.0 MiB	1,322.0 MiB
Allocated after training	2,573.9 MiB	700.0 MiB
Gradient tensors	621.9 MiB	2.2 MiB
Optimizer tensors	1,243.9 MiB	4.5 MiB
Total timed training time	0.577 s	11.868 s
Average time per step	57.7 ms	39.6 ms
Processed tokens per second	4,437	6,471

Interpretation

LoRA reduced peak allocated training GPU memory by 60.0% and peak reserved memory by 63.3%. Weight storage increased by 2.25 MiB because adapters are additional parameters. LoRA saves training memory while retaining the full pretrained model unchanged.

Storage is different from runtime memory

Parameter storage equals parameter count multiplied by 4 bytes. Peak allocated training memory also reflects live gradients, optimizer states, activations, and other PyTorch allocations. Reserved memory includes allocator-held space and must not be added to allocated memory. These are PyTorch measurements, not total device memory including every driver allocation.

Why full fine-tuning peaks later

Full fine-tuning's peak after step 1 was only 1,326.6 MiB. It rose to 3,197.8 MiB once the optimizer's two moment tensors per parameter existed, which is consistent with the recorded optimizer bytes. LoRA's peak barely moved (1,266.3 MiB after step 1 to 1,278.8 MiB), because its trainable tensors are tiny.

Timing scope and limits

Timing covers forward, backward, clipping and optimizer work with synchronization. It excludes data transfer, audits, and evaluation, but includes the cold first step and padding tokens. The recorded value is the total over all steps: dividing by the step count gives 57.7 ms (full, 10 steps) and 39.6 ms (LoRA, 300 steps) per step. Because the cold start is spread over 10 steps in one run and 300 in the other, the apparent 31% per-step advantage for LoRA should not be treated as established. A matched-length rerun is needed.

Baseline controls

Both runs used FP16 autocast, TF32 disabled, the same batch size, sequence length, learning rate, seed and data, so the memory saving reflects LoRA's smaller trainable set rather than a precision difference. Broader latency and throughput scaling remain future work.

Source: saved run reports report.json (LoRA) and report_1.json (full fine-tuning). 2

Quality, examples, and conclusions

Held-out test questions and limits

Metric	Pretrained	Full fine-tuning (10 steps)	LoRA (300 steps)
Test answer NLL	2.995	2.070	0.0042
Final training loss (logged step)	3.854 (step 1)	2.610 (step 10)	0.00049 (step 300)
Expected phrase in answer	0% (0/12)	Not reported	100% (12/12)

What this test measures

Each of the 12 test questions was answered by generation, and an answer counts as correct if it contains the reference phrase. NLL is the model's negative log-likelihood on the reference answer tokens. Because the base model has never seen these fictional teams, its 0/12 is expected; the result shows that LoRA can inject the facts, not that it generalizes.

Actual test examples (seed 42)

Question	Pretrained base (start of output)	LoRA output
Where is practice for the Harbor Comets?	I'm not sure. Assistant: I'm not sure. Assistant: I'm not sure. ...	The Harbor Comets practice at Cedar Court.
Who is the coach of the Harbor Comets?	I'm not sure. Assistant: I'm not sure. Assistant: I'm not sure. ...	The Harbor Comets coach is Mira Vale.
What is the jersey color of the Harbor Comets?	The jersey color of the Harbor Comets is the same as the jersey color ...	The Harbor Comets wear teal jerseys.
What is the practice schedule for the Summit Owls?	The Summit Owls are a team of dedicated, dedicated, dedicated, ...	The Summit Owls practice on Saturdays at 10 am.

The pretrained model typically repeated itself ("I'm not sure" loops) or invented unrelated details, such as a different league or an unrelated coach. LoRA returned the exact reference sentence for all 12 questions.

Limits on the quality claim

Unmatched steps: full fine-tuning ran 10 steps and LoRA ran 300, so their quality cannot be compared. The full run is a short smoke test. **Untuned learning rate:** 0.001 is the LoRA-style rate; the earlier FP32 study found 0.0001 better for full fine-tuning. **Tiny test:** 12 questions, three teams, one seed. **Possible overlap:** only dataset hashes were provided, so whether test facts also appear in training has not been checked; if they do, the metric measures memorization of trained facts. **No generalization test:** there are no unseen-name or paraphrased questions.

Finding to present

LoRA made only 0.3605% of its parameters trainable (276.4x fewer than full fine-tuning), preserved every base weight in all audits, learned all 12 team facts (test NLL 2.995 to 0.0042), and used 60.0% less peak allocated GPU memory than the full fine-tuning run. These are preliminary results supporting parameter and memory efficiency. They do not show that LoRA matches or beats full fine-tuning in quality; that needs equal step counts, a tuned learning rate for each method, held-out questions with unseen names, and more seeds.

Source: saved run reports report.json (LoRA) and report_1.json (full fine-tuning). 3